Building together

The DiracX Transformation System

DiracX hackathon · 12th DIRAC(X) Users' Workshop · FZU Prague



From a game to working code

TUE 16:10

Ice-breaker game

40 min

Run a workgraph by hand, with sticky notes.

TUE 16:50

1-2-4-All

1 h

What must exist, and how do we split the work?

WED 14:00

Mob programming

1 h 50

We build the database and the state machines together, with Claude.

WED 16:10

Planning poker

45 min

We size the work items and form pairs.

WED 17:00 → FRI

We build

about 6 h

Pairs and small pull requests. Demo on Friday.

> Code is becoming cheap.
> What we need is a **common understanding** of what to build, and of how we **review** it.

1 · Ice-breaker game

Run a Monte Carlo workgraph by hand

The game: 8 seeds become 1 merged file

Seed stack
8 red notes, S1 … S8

→

SIMULATION
1 seed per parcel

→

RECONSTRUCTION
2 green per parcel

→

MERGE
4 blue per parcel

→

Finished
1 yellow note `M1` : shout **“Completed!”**

ON EVERY BOARD, EACH PARCEL MAKES THE SAME TRIP

one per transformation: the board's player

shared by all transformations

Feeder → Pool → **Packer** → Ready → **Dispatcher** → **Compute backend** → **Completion** → ✓ Done
new notes → Outputs

✗ the note goes back to the Pool, a **new** parcel takes it

dark a player dashed a place on the table **Parcel**: a quarter sheet with its input notes stuck on it, one job

Round 1

ROUND

1

7 min

- Read your sheet. Ask your team, not us.
- Start when we say **go**.
- 8 seeds → 8 green → 4 blue → **1 yellow** `M1`.
- Shout **“Completed!”** when `M1` is in Merge's Outputs.

Round 2

ROUND

2

7 min

- Reset the table: notes back in the stack, new parcels.
- **Same game, same goal.**
- Shout **“Completed!”** when `M1` is in Merge's Outputs.

Round 3

ROUND

3

7 min

- **Change seats:** the observers come in, two players step out. Everyone else moves one seat to the left and takes the sheet in front of them.
- Reconstruction receives a **new version of its packer plugin**.
- Same goal. Shout **“Completed!”**

Debrief

What happened

- Where did you **wait**? Who was the bottleneck?
- When a job failed, what was **retried**: the parcel or the seed?
- How did you know the work was **finished**?
- Who knew how many notes were where? Now imagine **100 million** notes.

YOU PLAYED	WE WILL BUILD
Board players: feeding	**Feeder, edge feeder**
Board players: packing	**Packer**
Dispatcher, Compute backend	**Dispatcher, dummy backend**
Completion	**Completion: done, failed, split**
Nobody kept the books	**Database, state machines, counters**
The rules on the boards	**CWL submission**

> In the game, you moved the notes yourselves. In DiracX, a feeder or a packer only **says** what should happen. The core then updates the database in one step, so a note is never lost or counted twice.

2 · 1-2-4-All

What must exist, and how do we split it?

1-2-4-All

> What must exist for the whole Transformation System to work? What do we build first? What is unclear?

Already known: 2-week sprints · 4 to 5 pairs · the dummy executor exists and must be adapted to parcels · `simulation.cwl` in the simulator

1

`5 min · alone`

One piece of work per note 🟨, one question per note 🟥. In silence.

2

`10 min · in pairs`

Merge your notes, remove duplicates, sharpen the words.

4

`15 min · in fours`

Group into **work items**: one behaviour, one pair, one sprint. Say when it is **done**.

All

`25 min · together`

All items on the wall, in order, to the full system. Add only what is new. Pick the first sprint.

Outcome: the whole system in order; the first sprint as items, sized on Wednesday.

Example: *a failed parcel sends its inputs to Failed* · done: a test sees them Failed, ErrorCount 1 · needs: transitions

3 · Mob programming

The shared core, built together

Mob programming: the shared core, with Claude

WHAT WE BUILD

- database tables · DX-ADR-004
- state machines as data · DX-ADR-005
- transitions module: the only code that changes a status
- one transition + its test

1 · THE ROOM
Decides the next prompt
architects check it against the ADRs

→

2 · ONE PERSON
Types it into Claude
on the projector

→

3 · GIT
One prompt, one commit
pushed to a draft pull request

→

4 · EVERYONE
Reviews the diff live
and comments on GitHub

the comments become the next prompt

> Every pair will build on this code, so we learn it together.

4 · Planning poker

How big is each item?

Planning poker

1	2	3	5	8	13	?
Trivial. We know exactly what to change.	Small and clear.	Medium. Some thinking, few unknowns.	Large. Several parts, some unknowns.	Very large. Try to split it.	Too big. Split it before we start.	I do not understand the item.
 1 finger	 2 fingers	 3 fingers	 one hand	 5 + 3	 arms crossed	 fist

- Points measure **size**: effort, complexity, uncertainty. Not hours.
- “3, 2, 1, **show**”: everyone shows their fingers at the same time.
- The **highest** and the **lowest** explain. Then one more vote.
- Our usual pace: about **6 points per person per sprint**.

Now, let’s build it

- Pick your first item
- Small pull requests to `main`
- Review another pair's pull request before your next item
- Demo on Friday

Backup

Game material: set-up, table, role sheets

Game set-up and timing

One table of 8, two observers

- Sticky notes: red 🟥, green 🟩, blue 🟦 and yellow 🟨 pads (pink too, for the 1-2-4-All)
- 8 red notes written in advance: `S1` … `S8`
- Pens, scrap paper cut in quarters (the parcels)
- 3 boards, the role sheets (Compute backend: one per executor), the two cards on the last page
- 6 players: one of each role. 7 to 8: the Compute backend gets a second and a third executor.

TIME	STEP
3 min	**Explain the game**
5 min	**Read sheets, set up the table**
7 min	**Round 1, happy path**
2 + 7 min	**Reset, round 2, a surprise failure**
2 + 7 min	**Reset, round 3, new seats and new packer rule**
7 min	**Debrief**

The table

SIMULATION board

1 red per parcel

Pool	Done	Outputs

RECONSTRUCTION board

2 green per parcel

Pool	Done	Outputs

MERGE board

4 blue per parcel

Pool	Done	Outputs

Seed stack
S1 … S8, written in advance

Ready
parcels waiting for the Dispatcher

Backend seat
pads, pens; 1 to 3 executors

Bin
every used parcel, done or failed. Never reused.

A parcel is a quarter sheet with the input notes stuck on it (`P5 · RECO`). After the job: ✓ inputs go to Done, ✗ inputs go back to their Pool, the empty parcel goes to the Bin.

A note's name says where it comes from: `G3` is the green file made from seed `S3` ; `B1 = G3 + G5` .

Simulation

You play the Simulation transformation's two plugins: its **feeder** and its **packer**. Each transformation has its own.

You hold: the seed stack S1 … S8, paper quarters, a pen.

1. **Feed:** put seeds 🟥 from the stack into your Pool, two or three at a time.
2. When the stack is empty, say aloud: **“Simulation feed is over.”**
3. **Pack:** 1 seed per parcel. Write `P1 · SIM` (then P2…) on a paper quarter, stick the seed on it, put it on **Ready**.

Reconstruction

You play the Reconstruction transformation's feeder and packer. Your feeder is an **edge feeder**: its inputs are the outputs of Simulation.

You hold: paper quarters, a pen. Your packer rule may change in round 3.

1. **Feed:** move each green note from the Simulation Outputs into your Pool.
2. When Simulation's feed is over, its **Done holds all 8 seeds** and its Outputs is empty, say: **“Reconstruction feed is over.”**
3. **Pack:** 2 green per parcel (`P7 · RECO`), on Ready. Not enough notes? Wait. Once your feed is over, pack what is left.

Merge

You play the Merge transformation's feeder and packer. Your feeder is an **edge feeder**: its inputs are the outputs of Reconstruction.

You hold: paper quarters, a pen.

1. **Feed:** move each  blue note from the Reconstruction Outputs into your Pool.
2. When Reconstruction's feed is over, its **Done holds all 8 green notes** and its Outputs is empty, say: **“Merge feed is over.”**
3. **Pack:** 4 blue  per parcel (`P12 · MERGE`), on Ready. Not enough notes? Wait. Once your feed is over, pack what is left.

Dispatcher

In the real system: the part that hands each parcel to a backend. There can be several backends; the dispatcher chooses one.

You hold: nothing.

1. Take the oldest parcel on **Ready**.
2. Give it to an executor of the Compute backend that is **free**: no parcel in its hands.
3. If every executor is busy, wait with the parcel. Never give an executor two parcels.

Compute backend

In the real system: where jobs run. All three transformations send their jobs here; the boards only keep the books. You are one **executor**: one parcel at a time.

You share: the  pads, pens. If the backend gets a card, read it in secret.

1. **SIM** parcel: for seed `S3`, write a green 🟩 `G3` and stick it on the parcel.
2. **RECO** parcel: for two green notes, write one blue 🟦 `B1 = G3 + G5`.
3. **MERGE** parcel: for the blue notes, write one yellow 🟨 `M1 = B1 + B2 + B3 + B4`.
4. Write ✓ next to each input that worked (✗ if not), then give the parcel to the Completion.

Completion

In the real system: the step that reads the job's report and decides what happens to each input.

You hold: nothing.

1. ✓ input: put it in the **Done** column of its board. Put the new notes in **Outputs**.
2. ✗ input: put it back in the **pool** of its board. The Packer will make a new parcel.
3. Put the empty parcel in the **Bin**. A parcel is never reused.
4. When `M1` is in Merge's Outputs, every Pool and Ready are empty and nobody holds a parcel, shout **“Completed!”**

Simulation board

1 red per parcel · print on A4 landscape

Pool

waiting to be packed

In: Simulation (seeds), Completion (✗ inputs)
Out: Simulation, when packing

Done

processed by exactly one parcel

In: Completion only (✓ inputs)
Out: nobody

Outputs

green notes, for the next board

In: Completion only (new notes)
Out: Reconstruction takes the green notes

Reconstruction board

2 green per parcel · print on A4 landscape

Pool

waiting to be packed

In: Reconstruction (green notes), Completion (✗ inputs)
Out: Reconstruction, when packing

Done

processed by exactly one parcel

In: Completion only (✓ inputs)
Out: nobody

Outputs

blue notes, for the next board

In: Completion only (new notes)
Out: Merge takes the blue notes

Merge board

4 blue per parcel · print on A4 landscape

Pool

waiting to be packed

In: Merge (blue notes), Completion (✗ inputs)
Out: Merge, when packing

Done

processed by exactly one parcel

In: Completion only (✓ inputs)
Out: nobody

Outputs

the yellow `M1` : the finished product

In: Completion only (new notes)
`M1` stays here: the game is over

Cards

COMPUTE BACKEND · ROUND 2 · SECRET

A job fails

Parcel `P3 · SIM` fails: write ✗ next to its seed, make no green note, pass the parcel on as usual.

When that seed comes back on a new parcel, it works.

RECONSTRUCTION · ROUND 3

New packer version

3 green  per parcel, instead of 2.

Once your feed is over, pack what is left.